

Do we have a storage engine?

15 September 2026 · checked against `main` @ `2bc87ff` · companion to `VDB_Checklist.pdf` and `VDB_Feature_Implemented.pdf`

No — by design

VectoraDB has no storage engine of its own. Rows, indexes and the write-ahead log are stored by **PostgreSQL 16's own engine** on local disk. What VectoraDB built is a **storage layer around Postgres**: instant copy-on-write branches, space management, and durability to object storage.

How data is stored today

PostgreSQL 16 engine stock, unmodified	Heap tables, B-tree and other indexes, MVCC, write-ahead log, crash recovery. Runs in the <code>postgres:16-bookworm</code> image (plus wal-g) as one container per branch: <code>vec-main</code> , <code>vec-<branch></code> .	Used as is
▼ each branch's data directory lives on ▼		
Copy-on-write branch storage C1 · <code>internal/branch/storage.go</code>	One storage interface with two drivers. ZFS (macOS VM, Linux): pool on <code>/var/lib/vectoradb-zpool.img</code> , a branch is a snapshot + clone of <code>vectoradb/branches/main</code> . btrfs (Windows WSL): a branch is a subvolume snapshot. A branch of any size is created in seconds and only stores what changes.	Built, tested
Space management C6, C7	Reservation for main and optional per-branch caps (ZFS only); used / available shown by <code>vdb status</code> and the dashboard.	Partial
▼ main's changes are also copied, off the commit path, to ▼		
Durability to object storage D1–D3 · wal-g + MinIO	Continuous WAL archiving and base backups to a local S3-compatible store (MinIO), enabling point-in-time restore and "branch from just before a schema change" (H13). The commit never waits for this.	Built, tested
Own storage engine / networked storage not built	A custom engine or Neon-style storage service (page servers, WAL service) that Postgres reads from over the network. Listed in the README as "networked storage disaggregation" on the roadmap; nothing started.	Not started

Why no custom engine

The decision

- Transaction speed must never be compromised — it was the project's first locked requirement.
- Engines that separate compute from storage put a network hop under every page read and every commit.
- Stock Postgres on local disk keeps native latency; branching, backups and time travel are handled *beside* the commit path (copy-on-write file system, asynchronous WAL archiving).

What it means in practice

- Full PostgreSQL compatibility: drivers, ORMs and tools work unchanged, and extensions can be added to the image (pgvector isn't included yet).
- Nothing to maintain below Postgres — upgrades follow upstream.
- Limit: compute and storage run on one host. Scaling storage beyond a machine, or moving a branch between hosts, would need the networked storage that isn't built.

Where to find it

Piece	Code	Checklist / feature doc
Storage drivers (ZFS, btrfs)	<code>internal/branch/storage.go</code> ; driver chosen by <code>VECTORADB_STORAGE</code>	C1
Pool / filesystem provisioning	<code>internal/branch/provision.go</code>	C8
Branch lifecycle on top of it	<code>internal/branch/branch.go</code>	C2, C5
WAL archiving, backups, restore	<code>docker/postgres/Dockerfile</code> (wal-g), <code>internal/branch/branch.go</code> (Backup, Restore)	D1–D3, M6

VDB_Storage Engine · if the storage design changes, update this page, C1/D1 in both documents, and re-render with Chrome (see `make feature-doc`)